

Insight Generation

Melih Kayhan Pala

July 2026

1 Introduction

1.1 Background and Motivation

In an era defined by data-driven decision-making, the ability to extract meaningful and actionable insights from complex databases has become increasingly critical for commercial success[cite: 1]. Brands continuously accumulate massive volumes of customer interactions, feedback, and behavioral indicators across social media platforms. When meticulously preprocessed into structured formats—capturing key analytical dimensions such as sentiment, age, gender, and engagement metrics—this data harbors deep strategic value. However, the true potential of this structured social media intelligence often remains untapped due to the sheer volume of information and the complexity of identifying hidden correlations.

1.2 The Problem Statement

Generating insights traditionally requires significant manual effort[cite: 1]. Analysts must painstakingly preprocess, explore, and refine the data, which demands specialized domain knowledge to properly analyze and interpret the findings[cite: 1]. This manual bottleneck makes the process of hypothesis generation and validation both time-consuming and highly resource-intensive[cite: 1].

While there have been recent proposals for automating data analysis tasks using Large Language Models (LLMs), their outputs frequently fall short of human-crafted insights[cite: 1]. Existing tools typically operate only on single tables, require heavily pre-cleaned data, or depend entirely on user-defined goals, which severely limits their applicability to complex, real-world commercial use cases[cite: 1]. Furthermore, without a robust validation mechanism, LLMs struggle to process structured data accurately and often generate “shallow” observations—such as identifying simple outliers—rather than uncovering strategic business patterns[cite: 1].

1.3 The Proposed Framework

To bridge this gap, this project proposes a novel, automated framework designed specifically for the generation and validation of hypotheses from structured social media databases. Drawing upon state-of-the-art agentic architectures, our method leverages LLMs to automatically generate concise, text-based insights that reflect interesting and actionable patterns within the data[cite: 1].

The architecture shifts the paradigm from passive data querying to proactive intelligence gathering by simulating the reasoning process of an expert data analyst. The proposed pipeline operates through a comprehensive, three-step framework:

- **Hypothesis Generator:** This module formulates high-level, domain-relevant questions (e.g., brand perception trends across specific demographics) and breaks them down into precise, answerable sub-questions[cite: 1].
- **Query Agent:** Acting as the deterministic validation engine, this agent translates the generated sub-questions into executable SQL queries against the structured database to empirically test the hypotheses[cite: 1].
- **Summarization Module:** Finally, the quantitative results returned by the SQL queries are aggregated and verbalized into textual insights[cite: 1]. This step incorporates a rigorous reflection mechanism to filter out hallucinations, ensuring the final output is both correct and highly insightful[cite: 1].

1.4 Project Objectives and Commercial Impact

The ultimate objective of this project is to deliver a highly scalable, end-to-end analytical tool that transforms structured social media metrics into strategic foresight. By integrating automated, LLM-driven hypothesis generation with strict, SQL-based validation, the framework will empower decision-makers to rapidly discover and act upon hidden brand trends without writing a single line of code. This approach aims to drastically reduce the overhead associated with manual data analysis while guaranteeing that the generated insights are insightful, actionable, and strictly grounded in factual data[cite: 1].

2 Data Architecture and Preprocessing

2.1 Overview of the Data Architecture

The success of the proposed Text-to-SQL Large Language Model (LLM) pipeline fundamentally depends on the structural integrity of the underlying database. Social media data is inherently unstructured, noisy, and highly context-dependent. To enable the Query Agent to execute deterministic and accurate SQL queries, this raw data must first be transformed into a highly organized, relational

database schema. This section outlines the end-to-end data pipeline, from the ingestion of unstructured social media posts (e.g., X/Twitter data) to the formulation of a normalized, LLM-optimized database that captures multi-dimensional consumer intelligence.

2.2 Social Media Data Ingestion and NLP Preprocessing

Before the data enters the SQL environment, it passes through a rigorous preprocessing pipeline driven by Natural Language Processing (NLP) models. This phase is crucial for quantifying qualitative user interactions into queryable metrics.

- **Sentiment and Emotion Classification:** Raw texts are passed through sentiment analysis algorithms to be bucketed into core categories (Positive, Neutral, Negative). Beyond basic sentiment, advanced emotion detection models identify the “Dominant Emotion” of a post (e.g., Anger, Joy, Frustration).
- **Topic Modeling and Entity Extraction:** Named Entity Recognition (NER) and clustering algorithms group noisy, distinct conversations into standardized “Trending Topics” (e.g., mapped to macro-categories like “Customer Experience” or “Price/Value Perception”).
- **Demographic Inference:** Where available, metadata and linguistic cues are utilized to segment the data by key demographic indicators, primarily focusing on distinct “Age Groups” and “Gender”.
- **Consumer Journey Mapping:** Each social media interaction is classified into a specific stage of the consumer lifecycle (Consideration, Purchase, Recommendation, Complaint) using intent-recognition models.

2.3 Database Schema and Relational Modeling

To facilitate seamless query generation by the LLM, the preprocessed data is organized into a relational schema. This schema tracks both static volumes and dynamic momentum (growth/drop rates over time). The core architecture consists of the following interconnected tables:

- **brand_health_metrics Table:** This table captures high-level, aggregate indicators of brand performance. It tracks metrics such as *Share of Voice* (the brand’s visibility relative to competitors), *Emotional Bond*, *Brand Favorability*, *Net Promoter Score (NPS)*, and *Brand Advocacy*. It also stores historical momentum values to allow the LLM to query performance shifts over specific quarters.
- **emotion_analysis Table:** Quantifies the distribution of consumer feelings. It contains percentage-based volume columns for *Negative*, *Neutral*,

and *Positive* sentiments. Additionally, a *Dominant Emotion* string column allows the LLM to identify the most prevalent psychological trigger driving brand conversations at any given time.

- **consumer_journey Table:** This table maps interaction distribution across the customer lifecycle spectrum. It includes volume metrics for *Consideration*, *Purchase*, *Recommendation*, and *Complaint* stages. Crucially, this table calculates and stores meta-metrics such as the *Dominant Stage*, the *Fastest Growing* stage, and the stage experiencing the *Biggest Drop*, which are vital for generating automated strategic insights.
- **trending_topics Table:** Categorizes the primary drivers of brand conversation. It links granular, real-world discussion points (e.g., “Supply chain issues” or “Discount hunting”) to broader corporate categories, tracking the overall percentage volume and impact of each topic.
- **demographics Table:** This table serves as a relational bridge, allowing the system to execute complex cross-filtering. It correlates behavioral data with user profiles, specifically tracking *Emotions by Gender*, *Emotions by Age Groups*, and the distribution of demographics across the varying *Consumer Journey* stages.

2.4 LLM-Optimized Schema Design

A critical requirement for the Text-to-SQL architecture is the clarity of the database description. The schema is intentionally designed using highly descriptive, “verbose” column and table naming conventions rather than cryptic database acronyms. By providing the Hypothesis Generator and the Query Agent with an explicit schema and sample rows, the LLM can precisely understand the relationship between a user’s age group, their dominant emotion, and their current stage in the consumer journey. This structural clarity eliminates semantic ambiguity, ensuring that the SQL queries generated to validate hypotheses are syntactically and logically flawless.

3 Hypothesis (Insight) Generation

3.1 Architectural Motivation

The process of extracting valuable, automated insights directly from raw databases using Large Language Models (LLMs) presents significant technical hurdles. Standard LLMs possess limited context windows, making it impossible to feed them entire multi-table databases. Furthermore, despite their advanced natural language capabilities, LLMs still fundamentally struggle with processing and reasoning over raw structured data.

To bypass these limitations and avoid generating shallow, surface-level observations, this project adopts the multi-step Hypothesis Generator architecture

outlined in the reference literature. Rather than attempting to extract an insight in a single pass, this module simulates the cognitive workflow of an expert data analyst. It systematically creates overarching strategic questions and subsequently decomposes them into simpler, highly specific sub-questions that can be deterministically answered by querying the database.

3.2 The High-Level Generator (HL-G)

The first component of the hypothesis pipeline is the High-Level Generator (HL-G). The objective of the HL-G is to formulate broad, exploratory, and highly interesting business questions relevant to the brand’s commercial standing.

To achieve this, the HL-G is intentionally provided with only a shortened, high-level textual description of the database ($short(D_{info})$). By withholding the granular details of the database schema (such as specific column names, primary keys, or data types), the model is prevented from being artificially constrained by technical database mechanics. This design choice enables the LLM to generate more creative and strategically expansive questions within the domain.

Application Examples for HL-G:

Given the summary of the social media metrics database, the HL-G might formulate overarching commercial questions such as:

- **Business Question 1 (Brand Health & Logistics):** “How do supply chain disruptions and price sensitivity issues currently impact the emotional bond and recommendation rates for the brand across different demographics?”
- **Business Question 2 (Crisis Management):** “What specific demographic segments are driving the recent surge in ‘Anger’ and ‘Complaint’ stages, and which trending topics are most heavily associated with this negativity?”
- **Business Question 3 (Conversion Drivers):** “Is there a measurable correlation between the ‘Positive’ sentiment in the ‘Purchase’ stage and the volume of discussions around online shopping experience and satisfaction?”
- **Business Question 4 (Advocacy Momentum):** “Despite a 34% Share of Voice, why is the Net Promoter Score only at 16%, and what key journey stages are acting as bottlenecks for brand advocacy?”

3.3 The Low-Level Generator (LL-G)

While high-level questions are critical for strategic direction, they are generally too complex to be answered by a single database query. Therefore, the architecture employs a Low-Level Generator (LL-G) to bridge the gap between strategic intent and technical execution.

The LL-G takes the overarching question produced by the HL-G and processes it alongside the complete, verbose database description (D_{info}) and the exact structural schema (D_{schema}). By utilizing specific prompting techniques that expose the full intricacies of the tables (e.g., **brand_health_metrics**, **trending_topics**, **consumer_journey**), the LL-G is constrained to break the high-level question down into precise, concrete, and easily answerable sub-questions.

The LL-G is explicitly instructed to formulate these sub-questions so that they correspond to standard database operations (such as aggregations, sums, and averages). These sub-questions are designed to be non-sequential, allowing them to be executed in parallel during the validation phase.

Application Examples for LL-G:

Taking the high-level business questions generated above, the LL-G would parse the provided D_{schema} and generate executable sub-questions such as:

- *Deconstructing Business Question 1 (Brand Health & Logistics):*
 - “What is the percentage volume of the trending topic ‘Stok durumu ve tedarik zinciri aksaklıkları’ in the **trending_topics** table?”
 - “What is the current percentage of the ‘Complaint’ stage in the **consumer_journey** table, and what is its growth momentum?”
- *Deconstructing Business Question 2 (Crisis Management):*
 - “What is the highest percentage demographic in the **emotions_by_age_groups** table where the **dominant_emotion** is ‘Anger’?”
 - “Which topics in the **trending_topics** table have the highest percentage volume among users categorized in the ‘Negative’ sentiment bracket within the **emotion_analysis** table?”
 - “What is the numerical drop momentum for the ‘Recommendation’ stage in the **consumer_journey** table?”
- *Deconstructing Business Question 3 (Conversion Drivers):*
 - “What is the numerical volume of ‘Positive’ sentiment within the **emotion_analysis** table?”
 - “What is the percentage share of ‘Online alışveriş deneyimi ve memnuniyet’ in the **trending_topics** table?”
 - “What is the percentage of the ‘Purchase’ stage compared to the ‘Consideration’ stage in the **consumer_journey** table?”

4 Hypothesis Validation (Query Agent)

4.1 Conceptual Framework of the Validation Engine

Once the Hypothesis Generator has deconstructed broad business problems into targeted, granular sub-questions (s_{ij}), the system transitions from abstract rea-

soning to empirical validation. Directly prompting a standard Large Language Model (LLM) to answer these questions using data from memory introduces a high risk of hallucination and computational inaccuracy, particularly with complex numerical structures.

To enforce absolute factual grounding, this framework employs an agenic approach through a deterministic **Query Agent** (*QAgent*). The Query Agent acts as a translation layer that converts natural language sub-questions into precise, executable Structured Query Language (SQL) statements, executing them directly against the relational database (D). The mathematical objective of this agent is to synthesize a query (q_{ij}) whose resultant output table (R_{ij}) perfectly matches the semantic intent of the question, satisfying the relation:

$$q_{ij}(D) = R_{ij}$$

By leveraging SQL code execution instead of memory token prediction, the validation process relies entirely on the objective reality of the preprocessed database.

4.2 Text-to-SQL Generation and Self-Correction Loop

The Query Agent utilizes a multi-table orchestration system (such as LangGraph’s SQL Agent) to programmatically interface with the database. The workflow operates via a robust, execution-based self-correction loop:

1. **Context Assembly:** The agent receives the sub-question (s_{ij}), the verbose database descriptions (D_{info}), the explicit database schema (D_{schema}), and a small set of sample rows (typically 3 rows per table) to understand the exact data distribution and boundary conditions.
2. **Constraint Enforcement:** The agent is governed by strict system prompts. It is strictly prohibited from executing Data Manipulation Language (DML) statements (e.g., `INSERT`, `UPDATE`, `DELETE`) to ensure read-only safety. It is also instructed to query only the columns explicitly relevant to the question, avoiding resource-heavy wildcard (`SELECT *`) operations.
3. **The Self-Correction Mechanism:** Before delivering a final output, the agent is mandated to double-check its syntax. If database execution throws a runtime error (e.g., due to a mismatch in a categorical string or a syntax error in a complex join), the agent catches the error trace, feeds it back into its own prompt context, rewrites the query, and attempts re-execution. If a question cannot be resolved or falls outside the scope of the schema, the agent safely returns an “I don’t know” flag to prevent the propagation of erroneous data.

4.3 Table Verbalization and Semantic Filtering

The raw output of a successful SQL query is a relational table (R_{ij}), which is highly readable by machines but lacks the contextual narrative needed for

final business summaries. To bridge this gap, the Query Agent routes R_{ij} through a **Verbalization Module**. Using a specialized semantic prompt, an LLM translates the raw rows, counts, or averages back into structured natural language statements ($verb(R_{ij})$).

To guarantee the integrity of these verbalized statements, they are passed through an autonomous **LLM Evaluation Filter**. This filter scores each natural language answer based on two distinct axes:

- **Answerability:** Does the data retrieved actually satisfy the structural constraints of the question?
- **Relevance:** Is the answer logically aligned with the core hypothesis?

The module calculates an automated alignment score ($score_a$). In accordance with the reference literature, an empirical threshold of $\tau_a = 0.7$ is enforced. Any sub-question whose verbalized answer falls below this threshold is immediately discarded from the validation pipeline, ensuring that only high-quality, verified factual components move forward to the final summary phase.

4.4 Concrete End-to-End Validation Walkthroughs

To demonstrate the deterministic execution of the Query Agent, the following examples illustrate how the sub-questions generated in Section 3 are validated using the database schema from Section 2.

4.4.1 Example 1: Validation of Brand Health & Logistics Metrics

- **Input Sub-Question (s_{11}):** “What is the percentage volume of the trending topic ‘Stok durumu ve tedarik zinciri aksaklıkları’ in the `trending_topics` table?”
- **Agent Reasoning:** The agent recognizes the target table as `trending_topics` and identifies the conditional filter as the string matching the supply chain topic.
- **Generated SQL Query (q_{11}):**

```
SELECT topic_volume_percentage
FROM trending_topics
WHERE topic_name = 'Stok durumu ve tedarik zinciri aksaklıkları';
```

- **Database Result Table (R_{11}):**

topic_volume_percentage
16.0

- **Verbalized Answer ($verb(R_{11})$):** “The trending topic regarding supply chain and stock bottlenecks holds a conversation volume percentage of 16.0%.”
- **LLM Filter Evaluation:** $score_a = 0.98$ (Passes threshold $\tau_a \geq 0.7$).

4.4.2 Example 2: Validation of Demographic Crisis Metrics

- **Input Sub-Question (s_{21}):** “What is the highest percentage demographic in the `emotions_by_age_groups` table where the `dominant_emotion` is ‘Anger’?”
- **Agent Reasoning:** The agent notes that it needs to query `emotions_by_age_groups`, filter for the specific emotion ‘Anger’, sort the results in descending order based on volume, and isolate the top record.
- **Generated SQL Query (q_{21}):**

```
SELECT age_group, demographic_percentage
FROM emotions_by_age_groups
WHERE dominant_emotion = 'Anger'
ORDER BY demographic_percentage DESC
LIMIT 1;
```

- **Database Result Table (R_{21}):**

age_group	demographic_percentage
18-24	42.0

- **Verbalized Answer ($verb(R_{21})$):** “The demographic segment with the highest prevalence of anger is the 18-24 age group, accounting for 42.0% of the emotional distribution within that category.”
- **LLM Filter Evaluation:** $score_a = 0.95$ (Passes threshold $\tau_a \geq 0.7$).

4.4.3 Example 3: Handling Anomalous or Out-of-Scope Prompts (Filtering Phase)

- **Input Sub-Question (s_{33}):** “What is the average employee satisfaction score across store operations tables?”
- **Agent Reasoning:** The agent scans D_{schema} and D_{info} . It searches for tables related to employee metrics or internal corporate operations but finds only consumer-facing tables (`brand_health_metrics`, `emotion_analysis`, `consumer_journey`).
- **Agent Execution:** Because the question cannot be structurally mapped to the schema, the agent triggers its fallback constraint.
- **Generated Output:** “‘I don’t know’”
- **LLM Filter Evaluation:** $score_a = 0.00$ (Fails threshold $\tau_a = 0.7$; the question is discarded, preventing hallucinated filler text from contaminating the final business report).

5 Insight Summarization & Hallucination Filtering

5.1 The Role of the Summarization Module

The final stage of the architecture shifts the focus from structural data validation to natural language synthesis[cite: 1]. At this point in the pipeline, the Query Agent has successfully translated multiple sub-questions into SQL, executed them against the database, and produced a collection of disparate, verbalized facts ($verb(R_{i0}), \dots, verb(R_{im})$)[cite: 1]. While these individual facts are mathematically accurate, they exist as isolated fragments of information[cite: 1].

The **Summarization Module** acts as an executive-level translator[cite: 1]. Its primary goal is to ingest these individual verbalized statements, recognize the overarching macro-patterns connecting them, and synthesize a cohesive, strategic business insight (I)[cite: 1].

To maintain the highest level of executive engagement, the module is bound by strict formatting and stylistic constraints defined in the reference literature[cite: 1]:

- **Length Constraint:** The final insight must be concise and compressed into a maximum of 3 sentences[cite: 1]. Internal testing indicates that longer textual outputs suffer from a sharp drop in reader interest and operational effectiveness[cite: 1].
- **Pattern Over Enumeration:** The module is instructed to avoid a simple list or bulleted enumeration of facts[cite: 1]. Instead, it must reason through the data to express a high-level corporate pattern, using the most critical numerical metrics strictly as supporting evidence[cite: 1].
- **Actionable Grounding:** The insight must feel realistic and strategically valuable to decision-makers, avoiding generic platitudes while strictly refraining from introducing external, ungrounded recommendations[cite: 1].

5.2 The Hallucination Detection Framework ($score_h$)

Generative Large Language Models are inherently prone to text synthesis errors, such as introducing speculative biases, altering numerical values, or inventing external context—phenomena collectively known as hallucinations[cite: 1]. In a commercial decision-making environment, a single hallucinated metric can lead to catastrophic strategic missteps[cite: 1].

To eliminate this vulnerability, the system implements an automated **Hallucination Detector** before releasing any insight[cite: 1]. The detection engine operates through a two-step semantic auditing process:

1. **Claim Deconstruction:** The generated raw insight draft (I) is programmatically parsed and split into individual, atomic factual claims (C_i)[cite: 1].

- 1]. Each claim represents a distinct assertion that carries a binary truth value ($TV(C_i) \in \{0, 1\}$)[cite: 1].
2. **Contradiction Auditing:** An independent LLM evaluation prompt acts as a strict compliance auditor[cite: 1]. It systematically compares each atomic claim (C_i) against the original, verified text block provided by the Query Agent ($verb(R_{i0}), \dots, verb(R_{im})$)[cite: 1]. It looks for direct contradictions, ungrounded logical leaps, or modified numbers, ultimately computing a mathematical hallucination score ($score_h$)[cite: 1].

5.3 Iterative Reflection and Self-Correction (Algorithm 1)

To operationalize this audit, the system deploys an implementation of **Algorithm 1: Iterative Reflection Based on Verbal Relations**, leveraging a reinforcement learning paradigm known as *Reflexion*[cite: 1].

Algorithm 1: Iterative Reflection

```

1: i <- 0
2: while score_h(I, {verb(R_i0), ..., verb(R_im)}) >= \tau_h and i < maxit do
3:   reflect(I, {verb(R_i0), ..., verb(R_im)}) <- I
4:   i <- i + 1
5: end while
6: return I

```

The loop functions as a continuous optimization cycle:

- The system enforces a highly rigorous, experimentally verified hallucination threshold of $\tau_h = 0.9$ [cite: 1].
- If the calculated $score_h$ of the insight draft violates this threshold (indicating the presence of an ungrounded claim or a hallucination), the system triggers the **reflect** function[cite: 1].
- The model receives a reflection prompt containing the current summary draft, the failed score, and a detailed semantic explanation of *why* it failed (the critique)[cite: 1].
- The model uses this critique to re-evaluate the context, strip out the offending hallucinations, and generate an enhanced version of the summary[cite: 1].
- This cycle iterates continuously until either the insight achieves total compliance ($score_h < \tau_h$) or the loop hits a hard iteration ceiling (*maxit*), at which point it terminates to preserve computing efficiency[cite: 1].

5.4 Concrete End-to-End Summarization Walkthrough

The following walkthrough demonstrates how the Summarization Module processes the real-world outputs validated during the Query Agent phase in Section 4, detects a critical hallucination, and uses reflection to arrive at a flawless final insight[cite: 1].

5.4.1 1. Inputs Received from the Query Agent

- *verb(R₁₁)*: “The trending topic regarding supply chain and stock bottlenecks holds a conversation volume percentage of 16.0%.”
- *verb(R₂₁)*: “The demographic segment with the highest prevalence of anger is the 18-24 age group, accounting for 42.0% of the emotional distribution within that category.”
- *verb(R₂₂)*: “The ‘Complaint’ stage of the consumer journey has grown to encompass 55.0% of total interactions.”

5.4.2 2. Initial Insight Generation (Draft 1)

The module ingests these three facts and generates its first attempt at a strategic summary:

“Defacto is currently navigating a severe operational crisis, as supply chain disruptions and stock bottlenecks now dominate 16.0% of trending brand conversations. This logistics breakdown has directly driven customer complaints to an alarming 55.0% of the total consumer journey, resulting in a massive wave of anger concentrated specifically among male shoppers aged 18-24. We recommend that Defacto immediately fire its current regional logistics partner to salvage brand favorability.”

5.4.3 3. Factual Audit & Claim Deconstruction

The Hallucination Detector breaks Draft 1 down into four atomic claims and scores them against the original inputs[cite: 1]:

- **Claim 1:** *Supply chain bottlenecks account for 16.0% of trending topics.* → **True** (Matches *verb(R₁₁)*)[cite: 1].
- **Claim 2:** *Complaints represent 55.0% of the consumer journey.* → **True** (Matches *verb(R₂₂)*)[cite: 1].
- **Claim 3:** *Anger is heavily concentrated among male shoppers aged 18-24.* → **Hallucination!** (The source data *verb(R₂₁)* verified the 18-24 age demographic, but *never* isolated gender. The inclusion of the word “male” is an ungrounded inference)[cite: 1].

- **Claim 4:** *Defacto should fire its current regional logistics partner.* → **Hallucination!** (This is an prescriptive action step. The prompt guidelines strictly forbid adding recommendations that do not originate from the explicit data context)[cite: 1].

5.4.4 4. Critique & Reflection Cycle (Algorithm 1 Execution)

Because the ungrounded assertions push the hallucination score past the strict safety limit ($\tau_h = 0.9$), Draft 1 is rejected[cite: 1]. The system compiles a critique log:

- *Critique:* “Draft 1 falsely claims the angry demographic is explicitly male, which is not supported by the database. It also introduces an external operational recommendation regarding logistics partners that is entirely absent from the context. Rewrite the summary to be under 3 sentences, grounding it strictly in the numbers provided.”

5.4.5 5. Final Optimized Insight Output

The module processes the critique, strips out the ungrounded assumptions, and delivers a fully compliant, high-value strategic insight[cite: 1]:

“A major supply chain bottleneck is currently impacting Defacto, with stock and supply disruptions driving 16.0% of trending conversations. This operational issue strongly correlates with a critical emotional crisis among young consumers, as the 18-24 age group exhibits a dominant emotion of anger at 42.0%. This alignment suggests that inventory shortages are directly fueling demographic dissatisfaction, pushing overall customer complaints to a dominant 55.0% of the total consumer journey.”

6 Conclusion and Expected Commercial Impact

6.1 Summary of the Framework

This project outlines a robust, end-to-end architecture that successfully bridges the gap between unstructured social media noise and structured, deterministic business intelligence. By integrating Large Language Models (LLMs) with a rigid Text-to-SQL validation pipeline, the framework automates the historically manual and labor-intensive process of data analysis[cite: 1].

The system transitions the role of AI from a passive querying tool to an active, strategic reasoning engine. Through the orchestrated workflow of the Hypothesis Generator, the deterministic SQL execution of the Query Agent, and the rigorous hallucination filtering of the Summarization Module, the pipeline guarantees that abstract commercial questions are translated into mathematically verified, highly actionable insights[cite: 1]. By grounding the computational social science metrics (such as demographic sentiment and consumer jour-

ney momentum) in a strict relational database schema, the framework entirely eliminates the semantic ambiguity and hallucination risks typically associated with generative AI[cite: 1].

6.2 Expected Commercial Impact

The deployment of this automated hypothesis generation and validation tool in a live business analytics environment introduces several transformative commercial advantages:

- **Drastic Cost and Resource Optimization:** Traditional manual data analysis requires significant human capital, with industry estimates placing the cost of a single human-crafted insight at approximately \$140[cite: 1]. By automating the generation, validation, and summarization processes via API-driven LLM calls, this framework reduces the operational cost to roughly \$0.63 per insight[cite: 1]. This represents a near 99% cost reduction, allowing for massive scalability without increasing headcount.
- **Accelerated Time-to-Insight:** In fast-moving consumer markets, the delay between data collection and strategic action can be costly. This pipeline reduces the insight generation lifecycle from days or weeks to mere seconds. Executives can immediately identify brand crises (e.g., a sudden spike in “Anger” within the “Complaint” stage due to supply chain issues) and deploy targeted countermeasures before the momentum accelerates.
- **Democratization of Data Analytics:** By allowing stakeholders to interact with the system via high-level, natural language business questions, the tool removes the technical barrier of SQL proficiency. Marketing directors, product managers, and brand strategists can directly interface with complex social media metrics to extract customized intelligence without needing a dedicated data science intermediary.
- **Elimination of Analytical Bias and Hallucination:** Human analysts can inadvertently introduce confirmation bias when searching for trends, while standard LLMs are prone to hallucinating numbers to fit a narrative[cite: 1]. Because this system relies on a strictly deterministic SQL validation loop and an aggressive hallucination threshold ($\tau_h = 0.9$), the resulting strategic recommendations are completely objective and empirically grounded in the data[cite: 1].

6.3 Future Horizons

The immediate application of this framework successfully decodes social media interactions into brand health insights. However, the modular nature of the architecture allows for seamless future expansion. By continuously refining the hypothesis generation algorithms and integrating broader datasets—such as direct sales figures, macro-economic indicators, or competitor analysis—this pipeline is

positioned to become a foundational component of automated, enterprise-level business analytics.

7 References

References

- [1] R. Zhang and M. Elhamod, “Data-to-Dashboard: Multi-Agent LLM Framework for Insightful Visualization in Enterprise Analytics,” Boston University, Boston, MA, USA.
- [2] A. Vertsel and M. Rumiantsev, “Hybrid LLM/Rule-based Approaches to Business Insights Generation from Structured Data,” Narrative BI.
- [3] D. A. Pacheco and P. L. T. Alvarez, “Integration of Large Language Models with Power BI for Automatic Insight Generation,” *Scientific research article*, Universidad del Sinú Seccional Cartagena, Colombia, 2025. DOI: 10.17981/cesta.06.01.2025.06.
- [4] G. Sahu, A. Puri, J. Rodriguez, A. Abaskohi, et al., “INSIGHTBENCH: Evaluating Business Analytics Agents Through Multi-Step Insight Generation.”
- [5] B. Kühne and T. Böhm, “Formative Evaluation of Data-Driven Business Models – The Data Insight Generator,” University of Hamburg.
- [6] D. Wu, J. Li, M. Lu, G. Wang, and T. Chen, “Towards Autonomous Business Intelligence via Data-to-Insight Discovery Agent.”
- [7] A. Singha, B. Chopra, A. Khatry, S. Gulwani, et al., “Semantically Aligned Question and Code Generation for Automated Insight Generation,” Microsoft.
- [8] A. Sánchez Pérez, A. Boukhary, P. Papotti, L. Castejón Lozano, and A. Elwood, “An LLM-Based Approach for Insight Generation in Data Analysis,” Aily Labs, EURECOM.